# Java RadioButton

Java Swing

# JRadioButton

- We use the JRadioButton class to create a radio button.
- Radio button is use to select one option from multiple options.

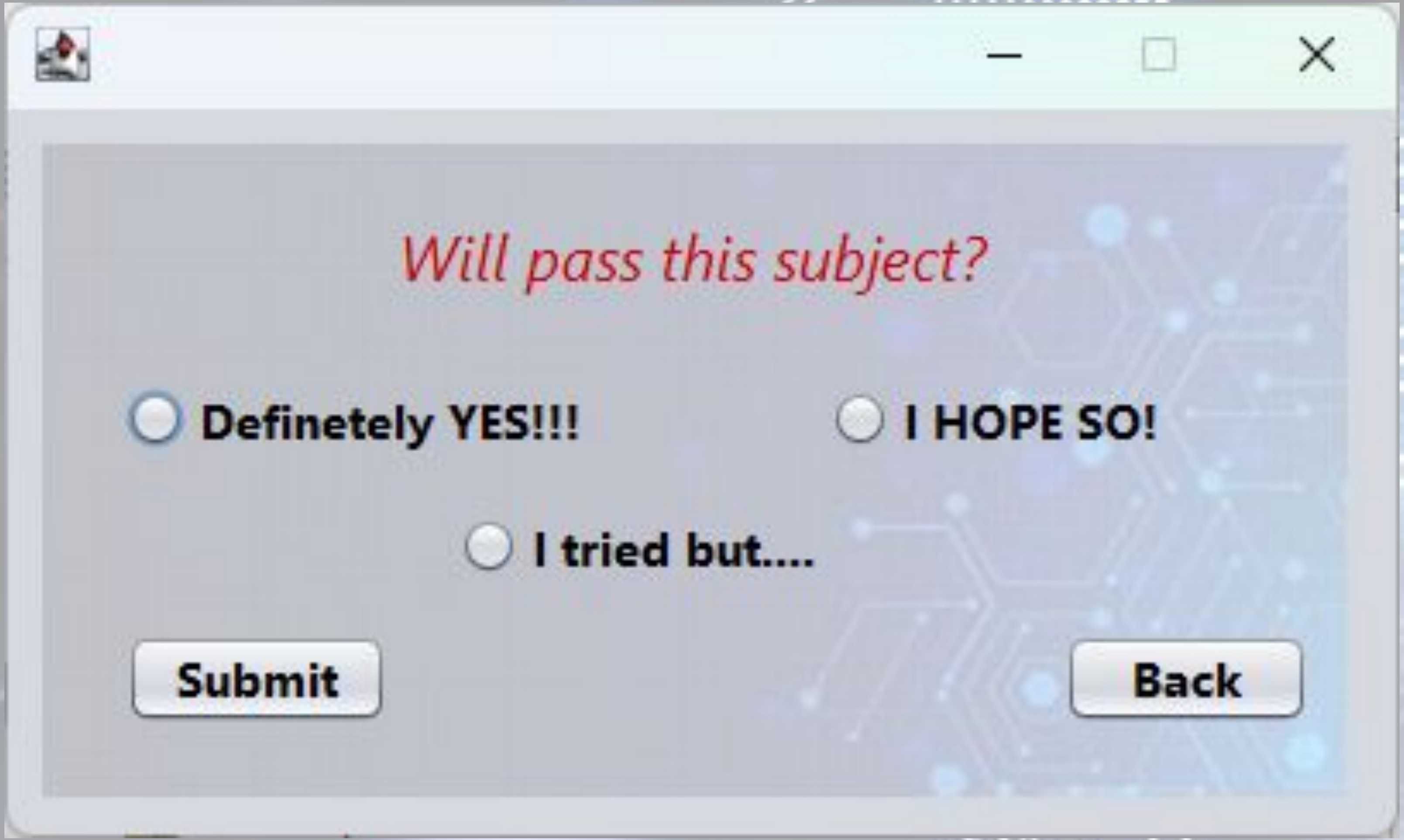
# Core Component Methods for JRadioButton

| Method | Description |
| --- | --- |
| **isSelected()** | Returns a boolean (true or false) indicating if the radio button is currently selected. |
| **setSelected(boolean b)** | Sets the radio button to a selected (true) or unselected (false) state programmatically. |
| **setText(String text)** | Sets the label or caption displayed next to the radio button. |
| **getText(** | Retrieves the current text label of the radio button. |
| **setEnabled(boolean b)** | Enables or disables the radio button for user interaction. |

# Example

- Create a Project (FinalProject).
- Create new jFrame (You can create a Main Menu Later)
- UI needs:
  - 1 Label
  - 3 RadioButton
  - 2 Button

# Sample Design



# Syntax - jRadioButtons

```
private void jRB1MouseClicked(java.awt.event.MouseEvent evt) {
    if (jRB1.isSelected()){
        jRB2.setSelected(false);
        jRB3.setSelected(false);
    }
}

private void jRB2MouseClicked(java.awt.event.MouseEvent evt) {
    if (jRB2.isSelected()){
        jRB1.setSelected(false);
        jRB3.setSelected(false);
    }
}

private void jRB3MouseClicked(java.awt.event.MouseEvent evt) {
    if (jRB3.isSelected()){
        jRB2.setSelected(false);
        jRB1.setSelected(false);
    }
}
```

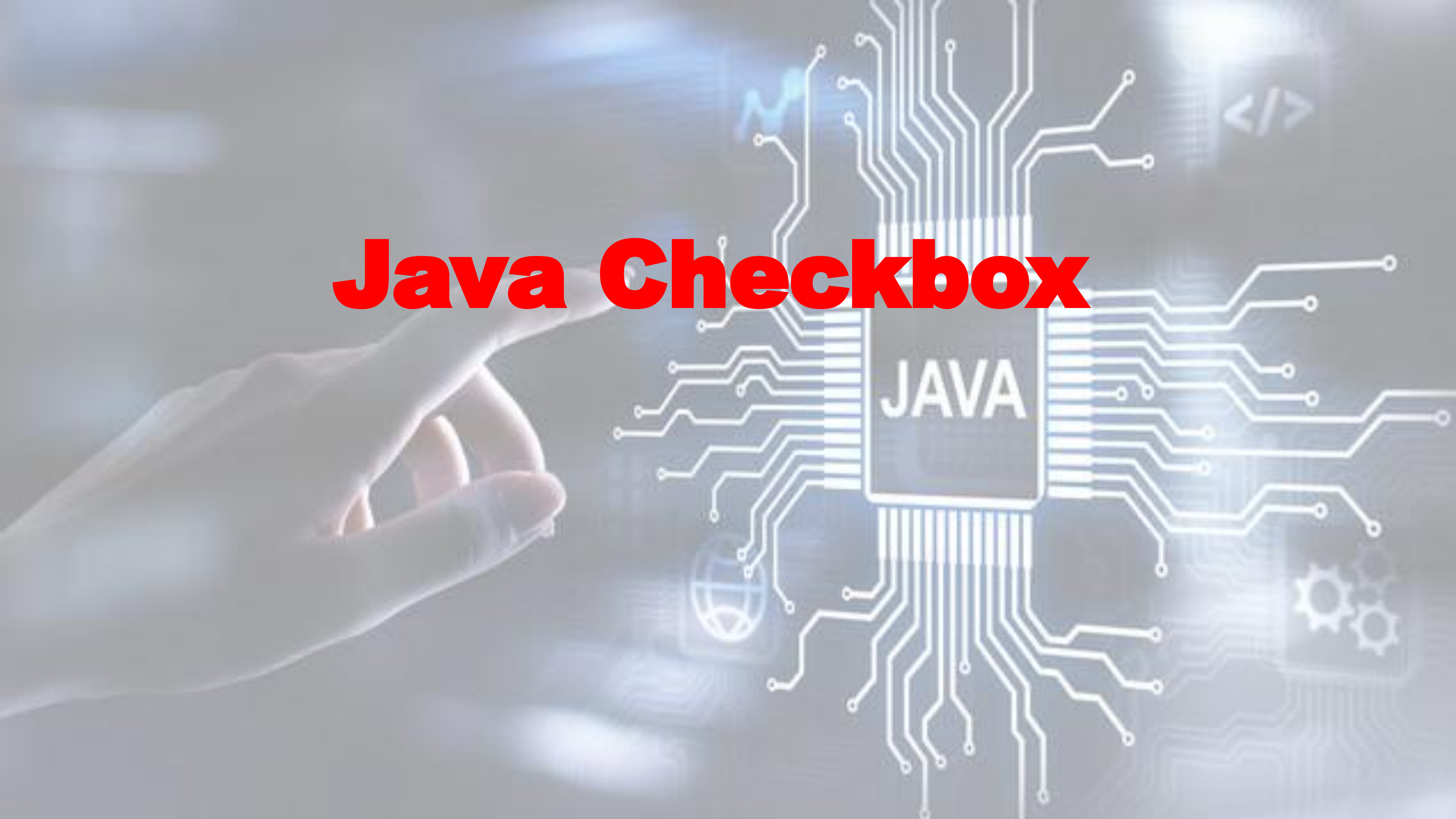
# jButton1 (Submit)

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    if (jRB1.isSelected()){
        JOptionPane.showMessageDialog(null, "Congratulations! Your hard work and dedication have paid off");
    } else if (jRB2.isSelected()) {
        JOptionPane.showMessageDialog(null, "I appreciate the effort you put into this project."
                + "Before we finalize it,"
                + "\nI think we need to adjust something on your project!");
    } else {
        JOptionPane.showMessageDialog(null, "Try and Try until you succeed!");
    }
}
```

# jButton2 (Back)

```java
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    this.setVisible(false);
    new MainMenu().setVisible(true);
}
```

# Java Checkbox



# JCheckBox - Java Swing

- JCheckBox is a part of Java Swing package.
- JCheckBox can be selected or deselected.
- It displays it state to the user .
- JCheckBox is an implementation to checkbox .

# Commonly used methods in JCheckBox

| Method | Description |
|---|---|
| **setText(String s)** | sets the text of the checkbox to the given text |
| **setIcon(Icon i)** | sets the icon of the checkbox to the given icon |
| **setSelected(boolean b)** | sets the checkbox to selected if boolean value passed is true or vice versa |
| **getIcon()** | returns the image of the checkbox |
| **getText()** | returns the text of the checkbox |

# jCheckBox Example

- Create new jFrame
- UI needs:
  - 1 CheckBox
  - 1 RadioButton
  - 1 Button

# jCheckBox Example

# JRadioButton Properties

- Text: Power
- Enabled: False (uncheck)
- Font: Any (Based on your preference)
- Horizontal Alignment: Center

# JCheckBox Properties

- Text: Power Switch
- Font: Any (Based on your preference)
-

# Syntax – Back Button

```java
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    this.setVisible(false);
        new MainMenu().setVisible(true);
}
```

# Import java awt

```java
package com.mycompany.finalproject;

import java.awt.Color;
```

# Syntax – CheckBox

```
private void jCheckBox1ActionPerformed(java.awt.event.ActionEvent evt) {
    if  (jCheckBox1.isSelected()) {
     jRadioButton1.setSelected(true);
     jRadioButton1.setForeground(Color.red);
     jRadioButton1.setText("Power is On");
    }else{
        jRadioButton1.setSelected(false);
     jRadioButton1.setForeground(Color.black);
     jRadioButton1.setText("Power is Off");
    }
}
```

# Activity #1

Simple Order System

# Syntax – Order Button

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    double Total;
    Total = Double.parseDouble(jSubTotal.getText());
    if (jCBH1.isSelected()){
        Total = Total + 65;
        jSubTotal.setText(String.valueOf(Total));
        jCBH1.setSelected(false);
    } if (jCBH2.isSelected()) {
      Total = Total + 65;
      jSubTotal.setText(String.valueOf(Total));
      jCBH2.setSelected(false);
    } if (jRB1.isSelected()) {
      Total = Total + 35;
      jSubTotal.setText(String.valueOf(Total));
      jCBT3.setSelected(false);
    } if (jRB2.isSelected()) {
      Total = Total +45;
      jSubTotal.setText(String.valueOf(Total));
      jCBT3.setSelected(false);
    } if (jRB3.isSelected()) {
      Total = Total + 45;
      jSubTotal.setText(String.valueOf(Total));
      jCBT3.setSelected(false);
    }
}
```

# Java List

Java Swing

# What is JList

- It is part of Java Swing package.
- It is a component that displays a set of Objects and allows the user to select one or more items.
- It inherits JComponent class.
- It is an easy way to display an array of Vectors .

# Constructor for JList are:

- **JList() -** Creates an empty blank list
- **JList(E [ ] l) -** Creates an new list with the elements of the array.
- **JList(ListModel d) -** creates a new list with the specified List Model.
- **JList(Vector l)** - creates a new list with the elements of the vector

# Commonly used methods are :

| Methods | Description |
|---|---|
| **getSelectedIndex()** | returns the index of selected item of the list. |
| **getSelectedValue()** | returns the selected value of the element of the list |
| **setSelectedIndex(int i)** | sets the selected index of the list to i |
| **setSelectionBackground(Color c)** | sets the background Color of the list |
| **setSelectionForeground(Color c)** | Changes the foreground color of the list |

# Commonly used methods are :

| Methods | Description |
|---|---|
| **setListData(E [ ] l)** | Changes the elements of the list to the elements of l . |
| **setVisibleRowCount(int v)** | Changes the visibleRowCount property |
| **setSelectedValue(Object a, boolean s)** | selects the specified object from the list. |
| **setSelectedIndices(int[] i)** | changes the selection to be the set of indices specified by the given array. |

# Commonly used methods are :

| Methods | Description |
|---|---|
| **setListData(Vector l)** | constructs a read-only ListModel from a Vector specified. |
| **setLayoutOrientation(int l)** | defines the orientation of the list |
| **setFixedCellWidth(int w)** | Changes the cell width of list to the value passed as parameter. |
| **setFixedCellHeight(int h)** | Changes the cell height of the list to the value passed as parameter. |
| **isSelectedIndex(int i)** | returns true if the specified index is selected, else false. |

# Commonly used methods are :

| Methods | Description |
| --- | --- |
| **indexToLocation(int i)** | returns the origin of the specified item in the list's coordinate system. |
| **getToolTipText(MouseEvent e)** | returns the tooltip text to be used for the given event. |
| **getSelectedValuesList()** | returns a list of all the selected items. |
| **getSelectedIndices()** | returns an array of all of the selected indices, in increasing order |
| **getMinSelectionIndex()** | returns the smallest selected cell index, or -1 if the selection is empty. |

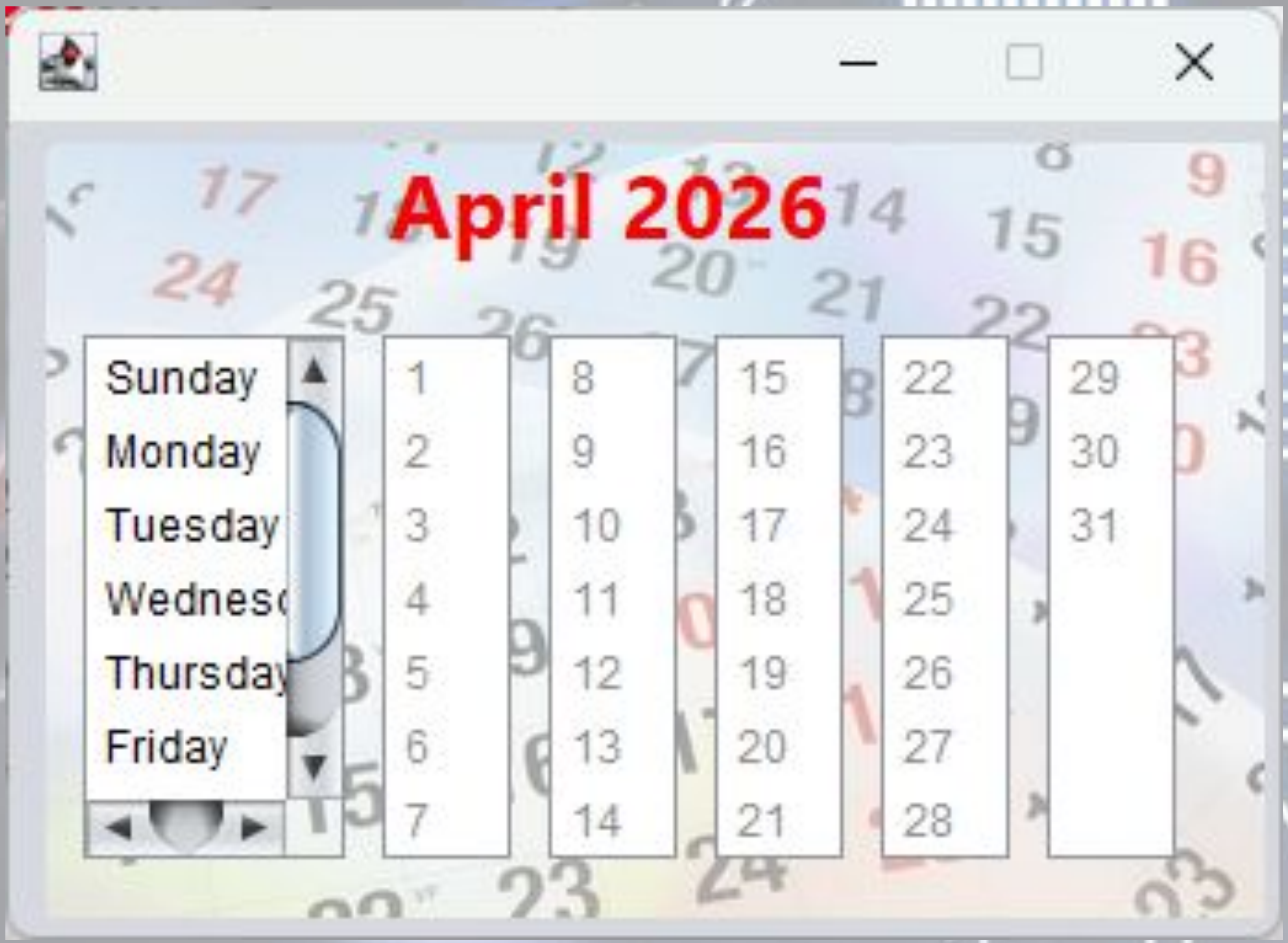
# Commonly used methods are :

| Methods | Description |
|---|---|
| **getMaxSelectionIndex()** | returns the largest selected cell index, or -1 if the selection is empty. |
| **getListSelectionListeners** | returns the listeners of list |
| **getLastVisibleIndex()** | returns the largest list index that is currently visible. |
| **getDragEnabled()** | returns whether or not automatic drag handling is enable |

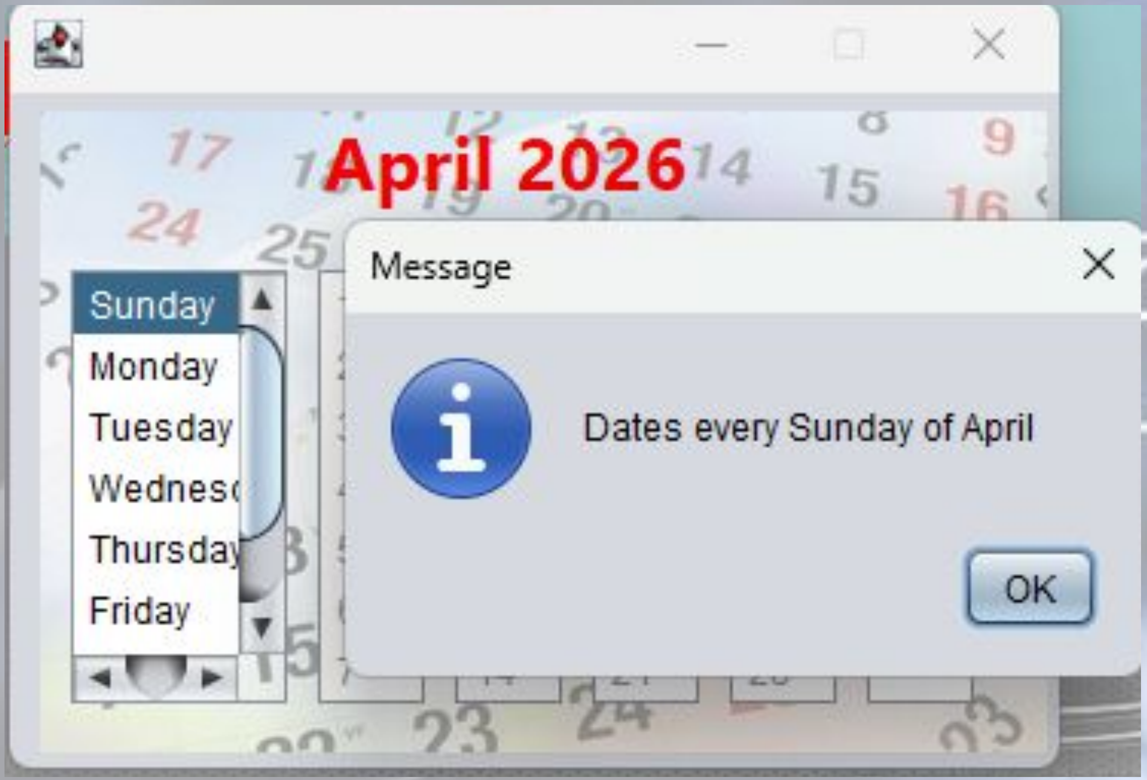
# Commonly used methods are :

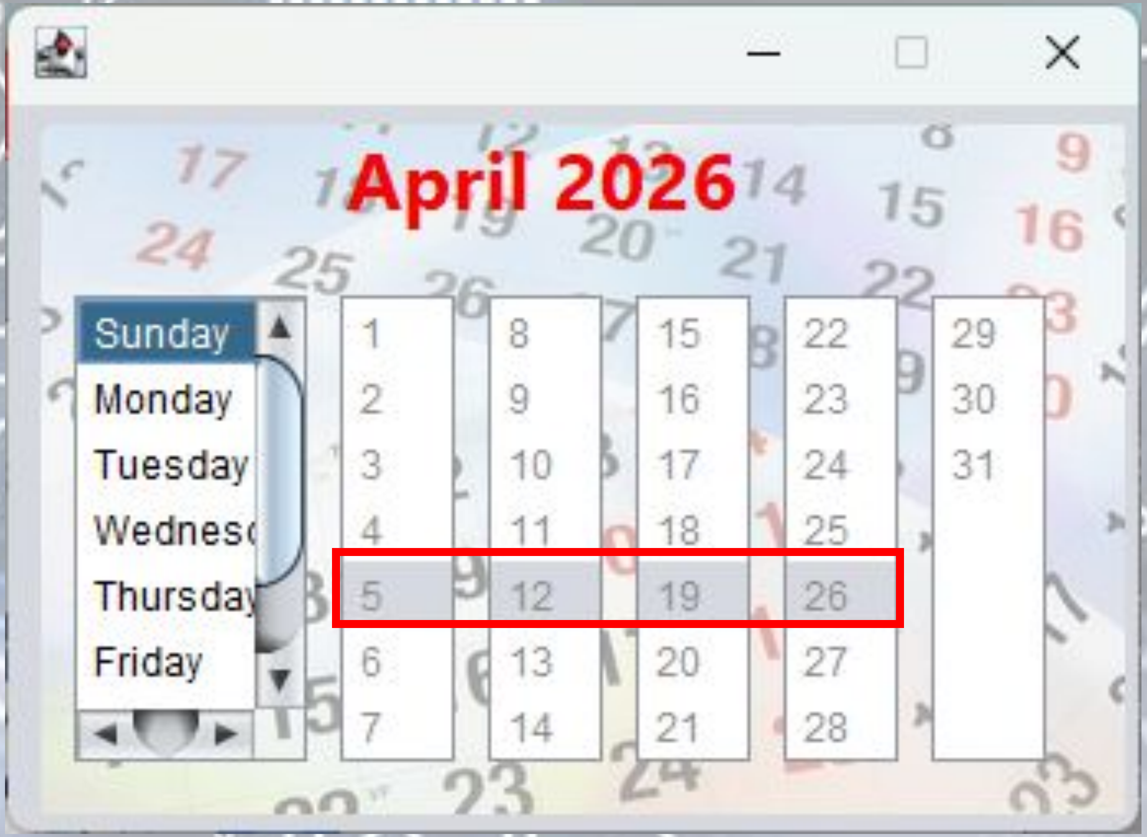
| Methods | Description |
| --- | --- |
| **addListSelectionListener(ListSelectionListener l)** | adds a listSelectionlistener to the list |

# jList Example

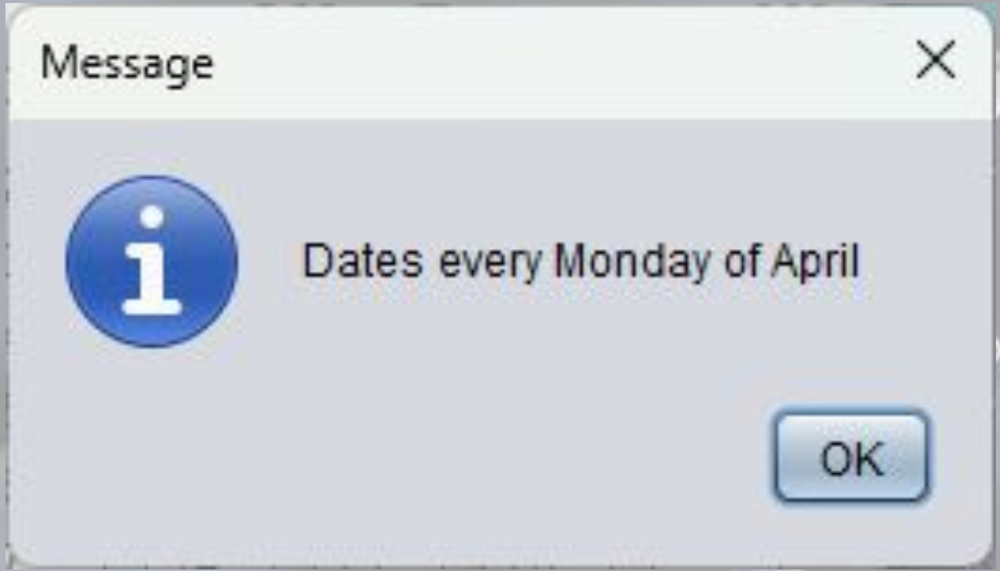


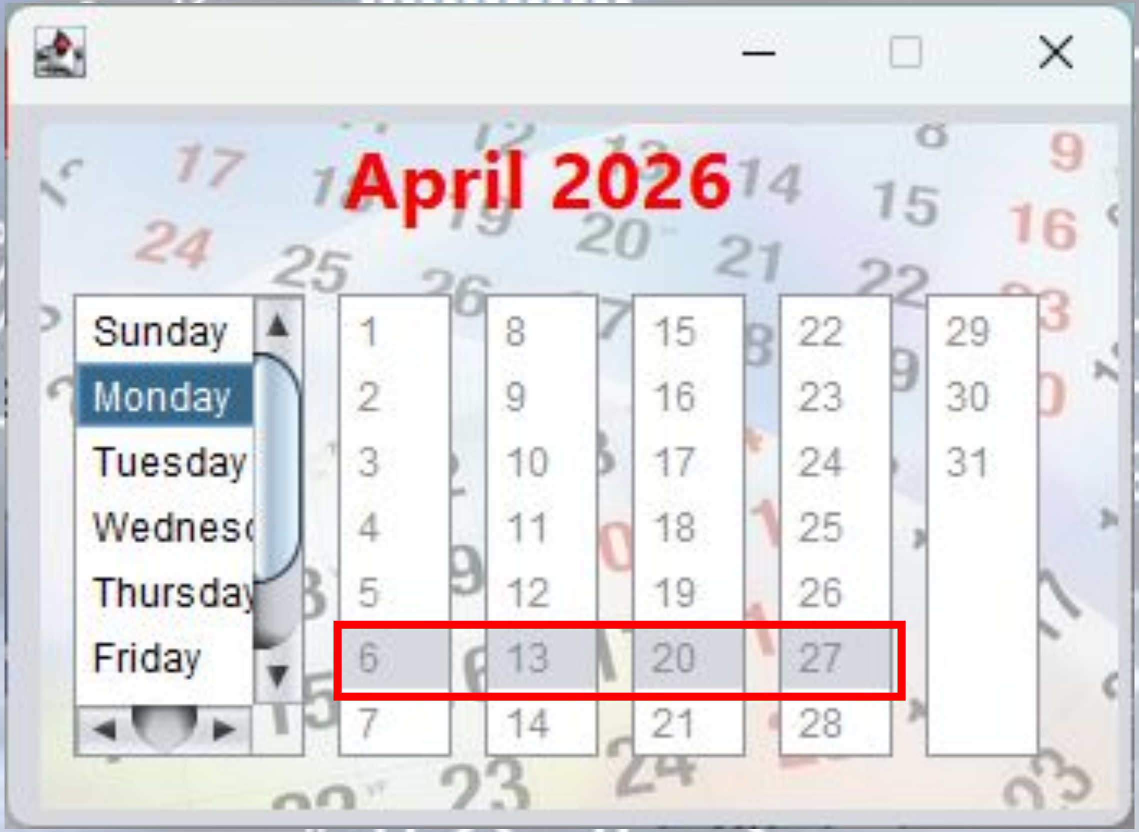
# jList Example



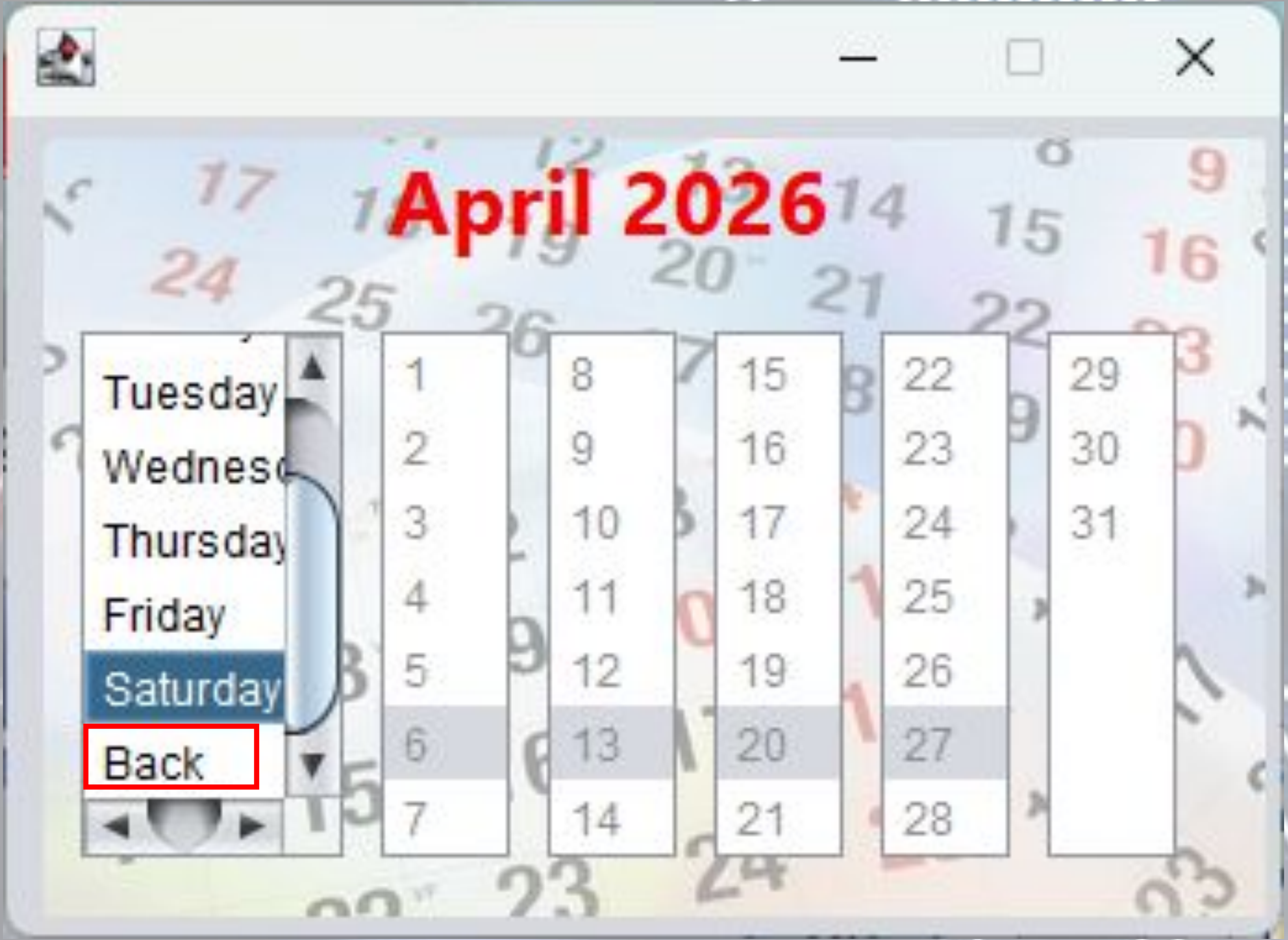



# jList Example



# jList Example



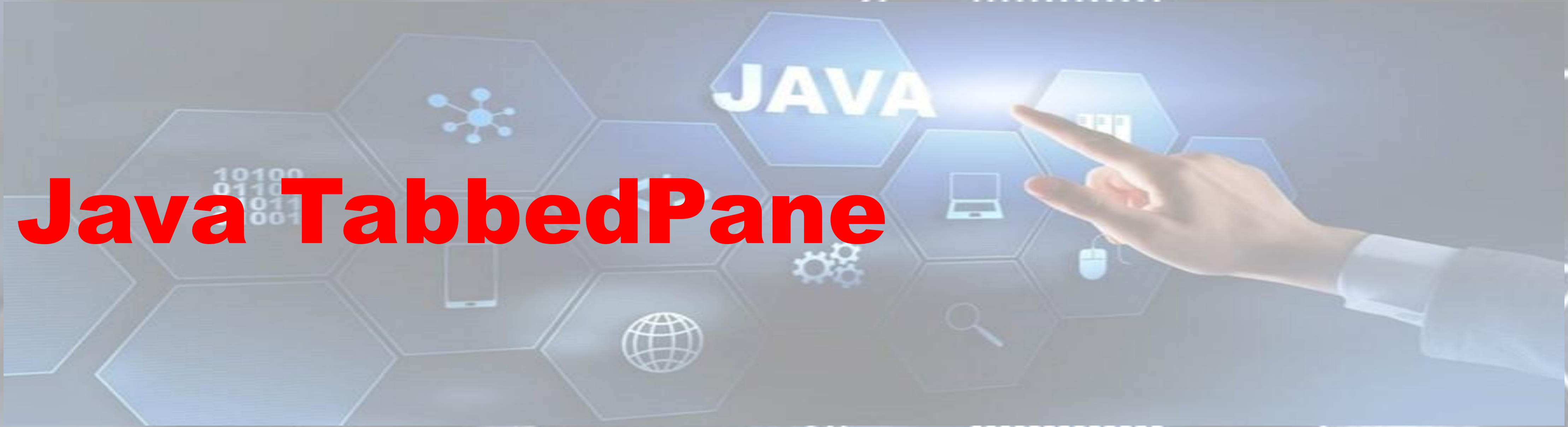
# Syntax

```
private void jList1MouseClicked(java.awt.event.MouseEvent evt) {
    if (jList1.isSelectedIndex(0)) {
       JOptionPane.showMessageDialog(null, "Dates every Sunday of April");
        jList2.setSelectedIndex(4);
        jList3.setSelectedIndex(4);
        jList4.setSelectedIndex(4);
        jList5.setSelectedIndex(4);
    } else if (jList1.isSelectedIndex(1)) {
        JOptionPane.showMessageDialog(null, "Dates every Monday of April");
        jList2.setSelectedIndex(5);
        jList3.setSelectedIndex(5);
        jList4.setSelectedIndex(5);
        jList5.setSelectedIndex(5);
```

# Syntax

```
} else if (jList1.isSelectedIndex(7)) {
    jList1.getToolTipText();
    this.setVisible(false);
    new Seatwork2().setVisible(true);
}
```

# Java TabbedPane




Java Swing

# What is JTabbedPane?

- It is a GUI(Graphical User Interface) component in the Java Swing library that allows you to create a tabbed pane interface.
- It is a container that can store and organize multiple components into distinct tabs. It contains a collection of tabs.

# Key Features of JTabbedPane

- **Tabs:** Each tab in a JTabbedPane is normally represented by a title or icon, and clicking on a particular tab shows the information only related to that tab.
- **Tab Placement**: JTabbedPane provides many tab placement options, such as top, bottom, left, or right sides of the pane, allowing you to customize the layout.

# Key Features of JTabbedPane

- **Scrolling:** If the tabs don't fit in the visible region, JTabbedPane provides scrolling options to navigate through them.
- **Event Handling:** You can add event listeners to JTabbedPane to respond to tab selection or deselection events, allowing you to do actions when the user switches between tabs.

# Constructors used in JTabbedPane

| CONSTRUCTOR | DESCRIPTION |
| --- | --- |
| **JTabbedPane()** | The JTabbedPane class in Java Swing has a default, no-argument constructor called JTabbedPane(). This constructor initializes an empty tabbed pane with no tabs and no initial content when a JTabbedPane is created. |

# Constructors used in JTabbedPane

| CONSTRUCTOR | DESCRIPTION |
|---|---|
| **JTabbedPane(int tabPlacement)** | The JTabbedPane(int tabPlacement) constructor allows to creation of a JTabbedPane with a defined initial location for the tabs. The tab placement option specifies whether the tabs appear at the top, bottom, left, or right of the tabbed pane. |

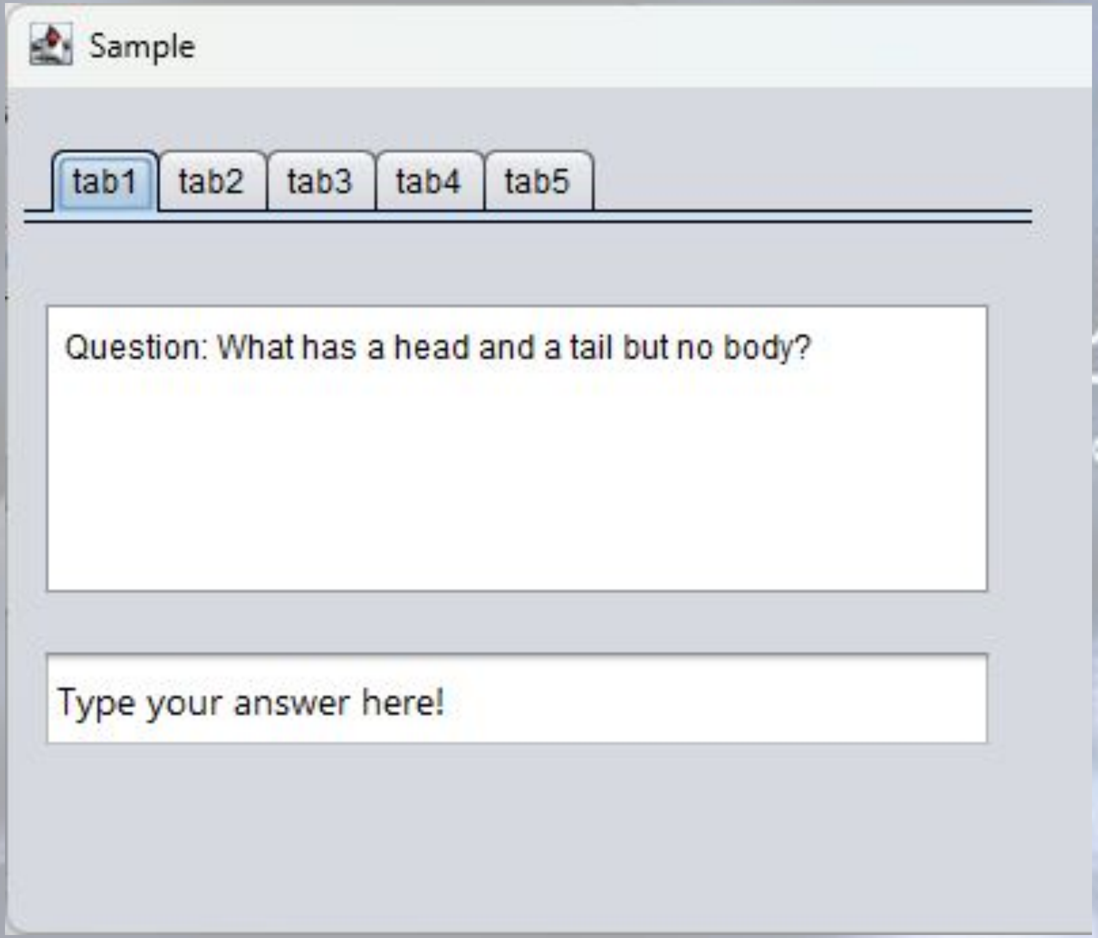
# Some commonly used methods of the JTabbedPane

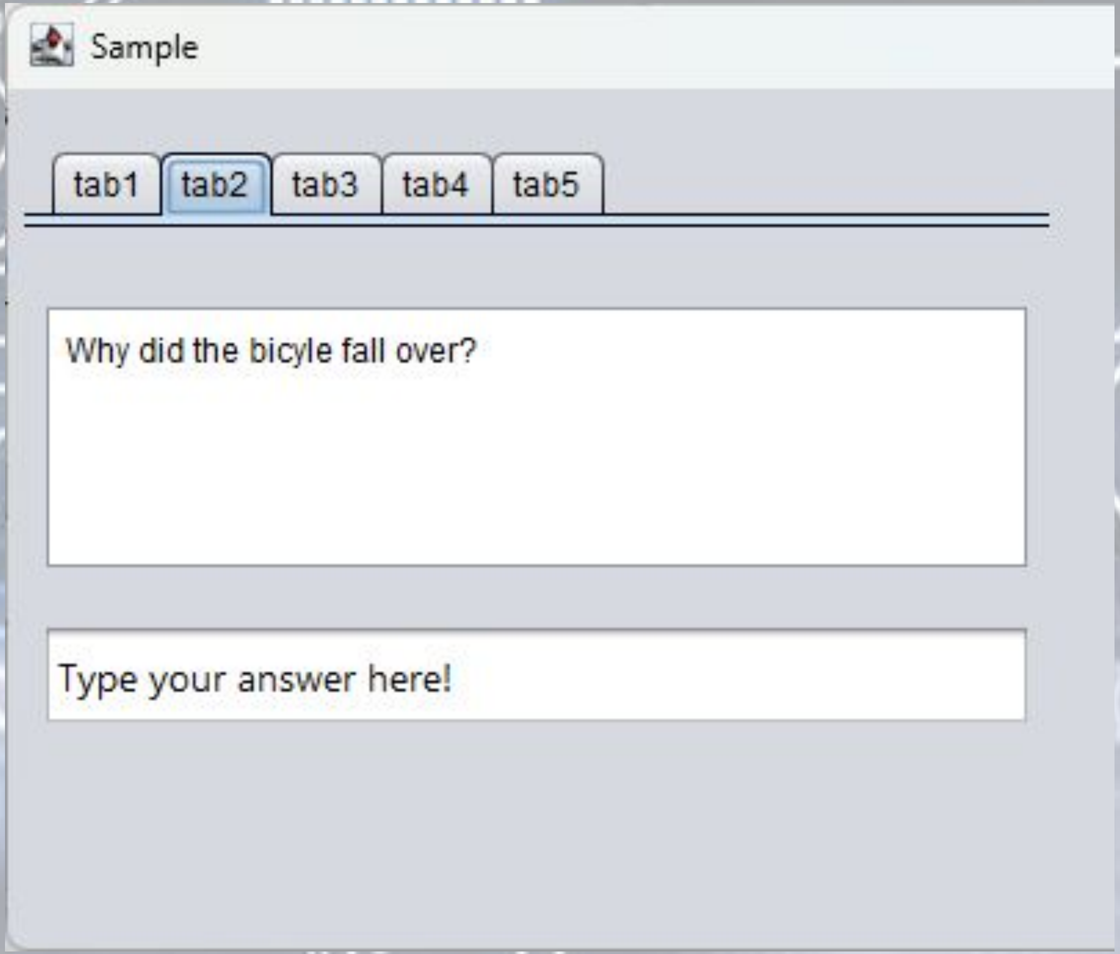
| Method | Description |
|---|---|
| **addTab(String title, Component component)** | Creates a new tab with the given title and content. |
| **removeTabAt(int index)** | Removes the tab at the given index. |
| **getTabCount()** | Returns the number of tabs present in the JTabbedPane. |

# Some commonly used methods of the JTabbedPane

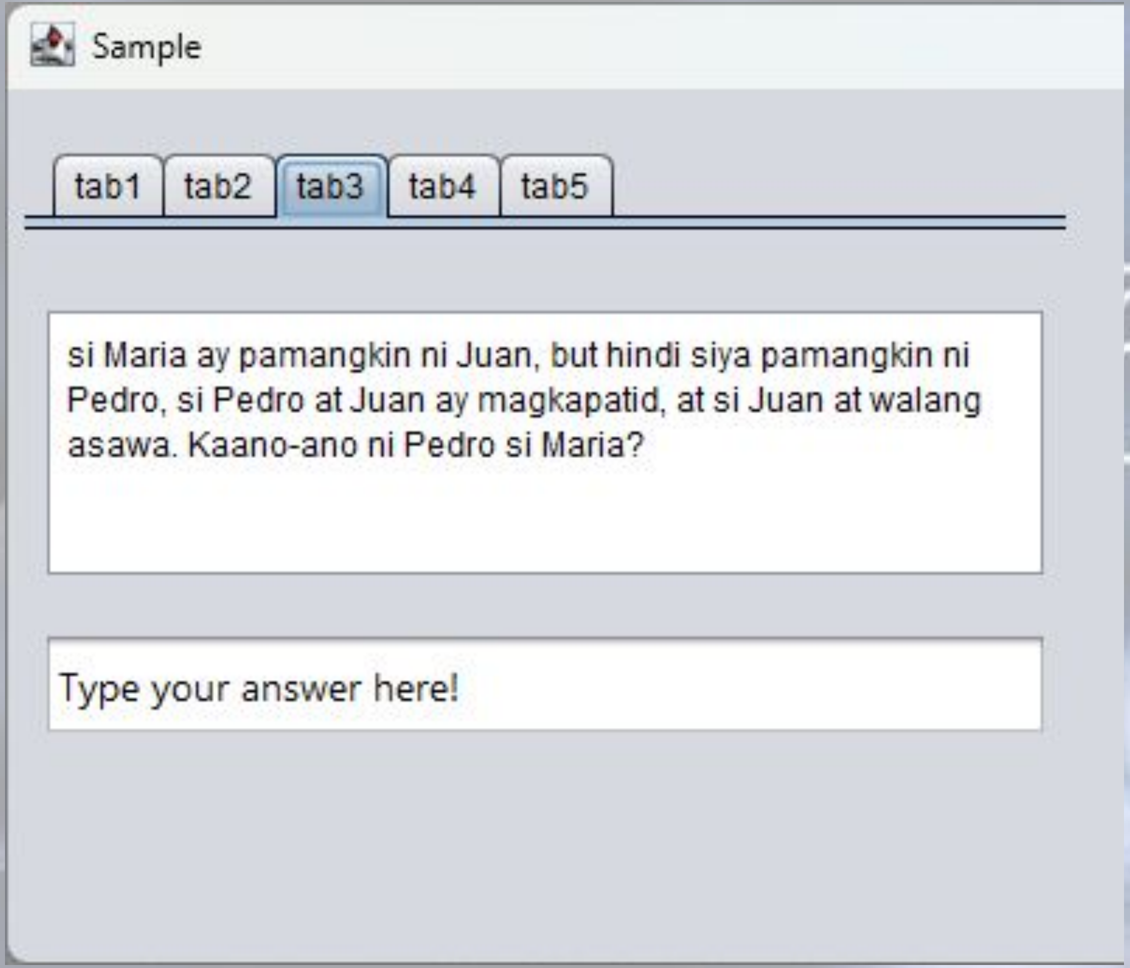
| Method | Description |
|---|---|
| **setSelectedIndex(int index)** | Sets the chosen tab to the index given. |
| **getSelectedIndex()** | Returns the index of the currently selected tab. |

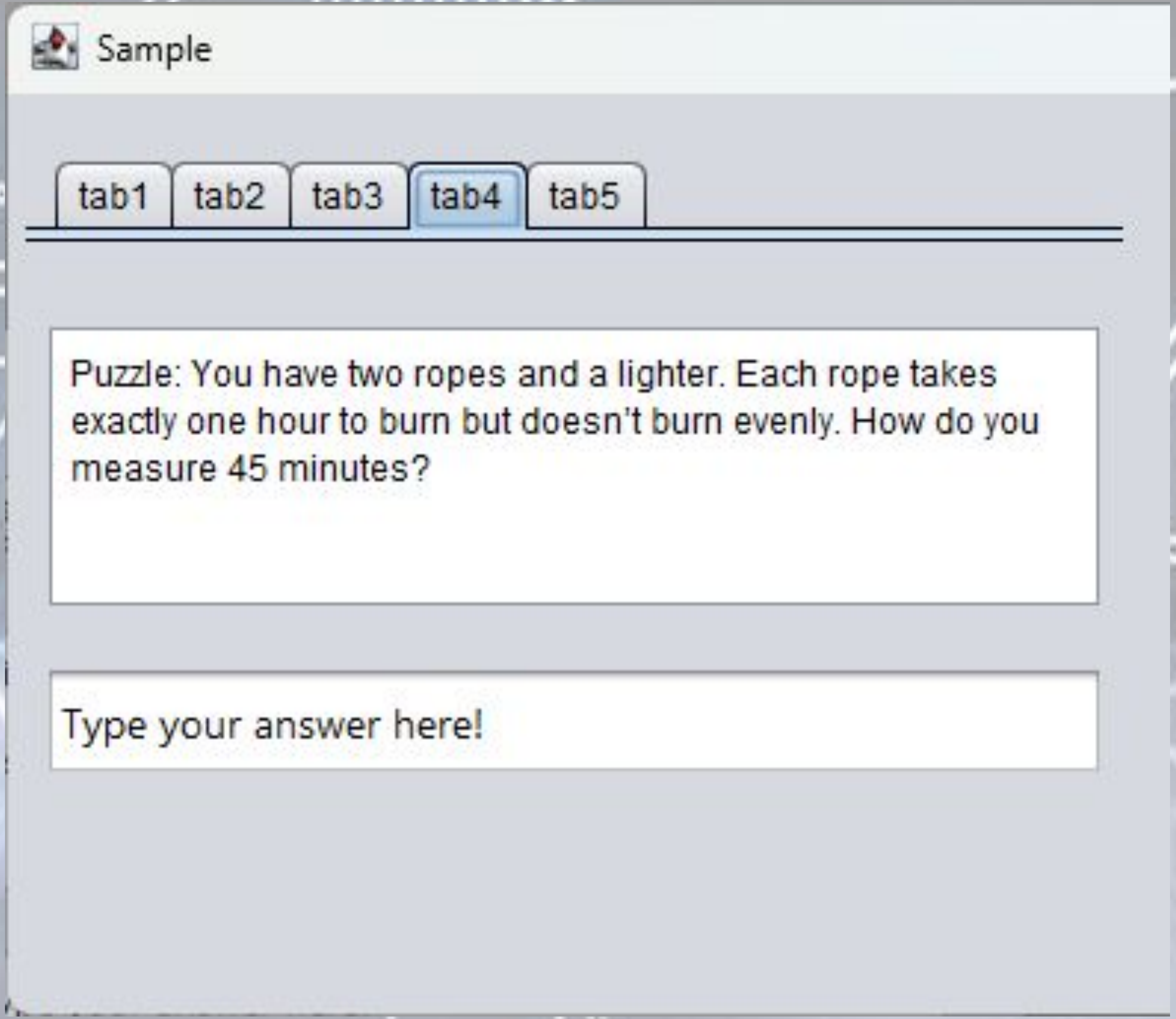
# JTabbedPane Example



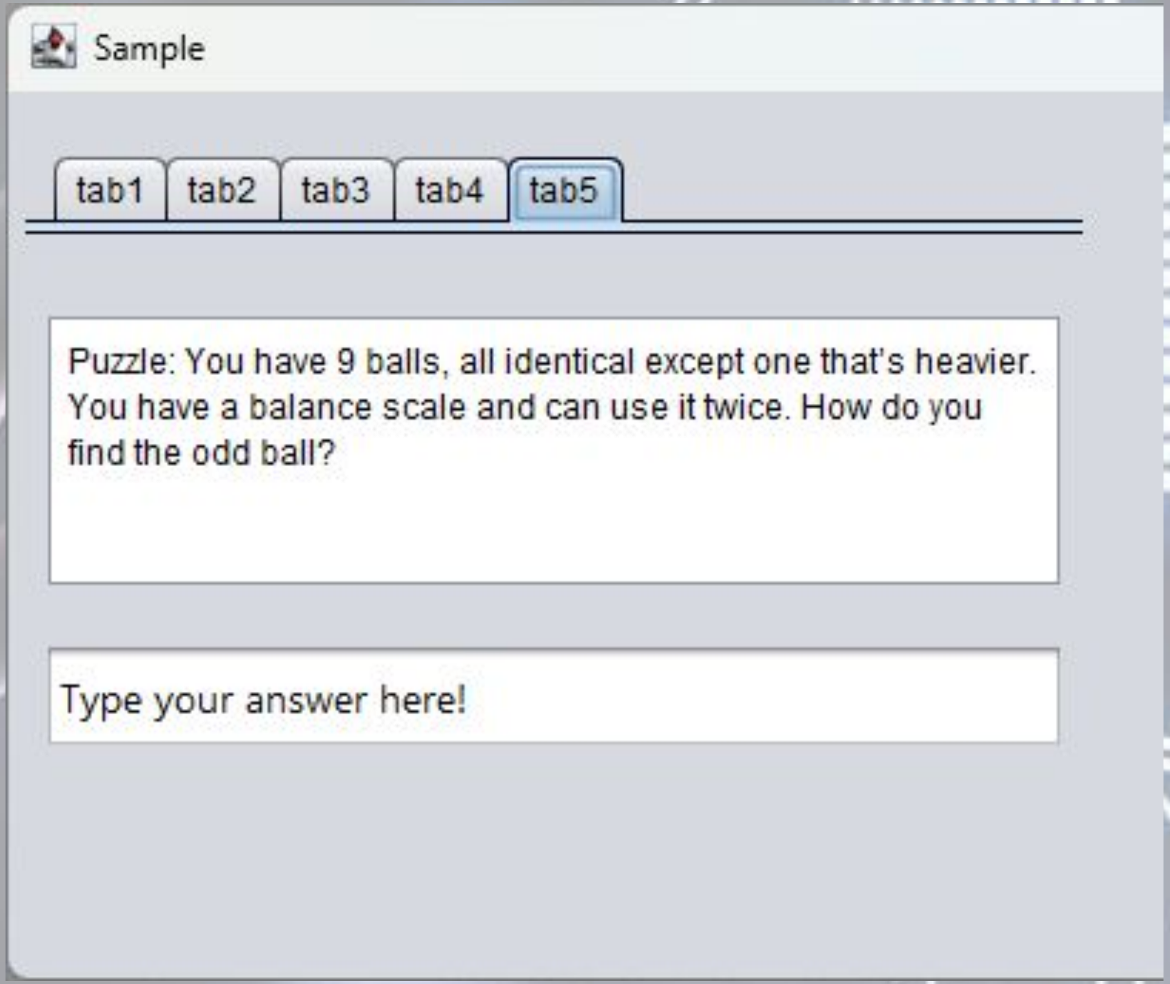



# JTabbedPane Example



# JTabbedPane Example



# Syntax

```
private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {
    String  answer1 = jTextField1.getText();
    if (answer1.equalsIgnoreCase("Coin")) {
        JOptionPane.showMessageDialog(null, "Your Answer is Correct");
       jTextField1.setEnabled(false);
    } else {
        JOptionPane.showMessageDialog(null, "Your Answer is incorrect.\n Try Again!");
        jTextField1.setText("");
    }
}
```

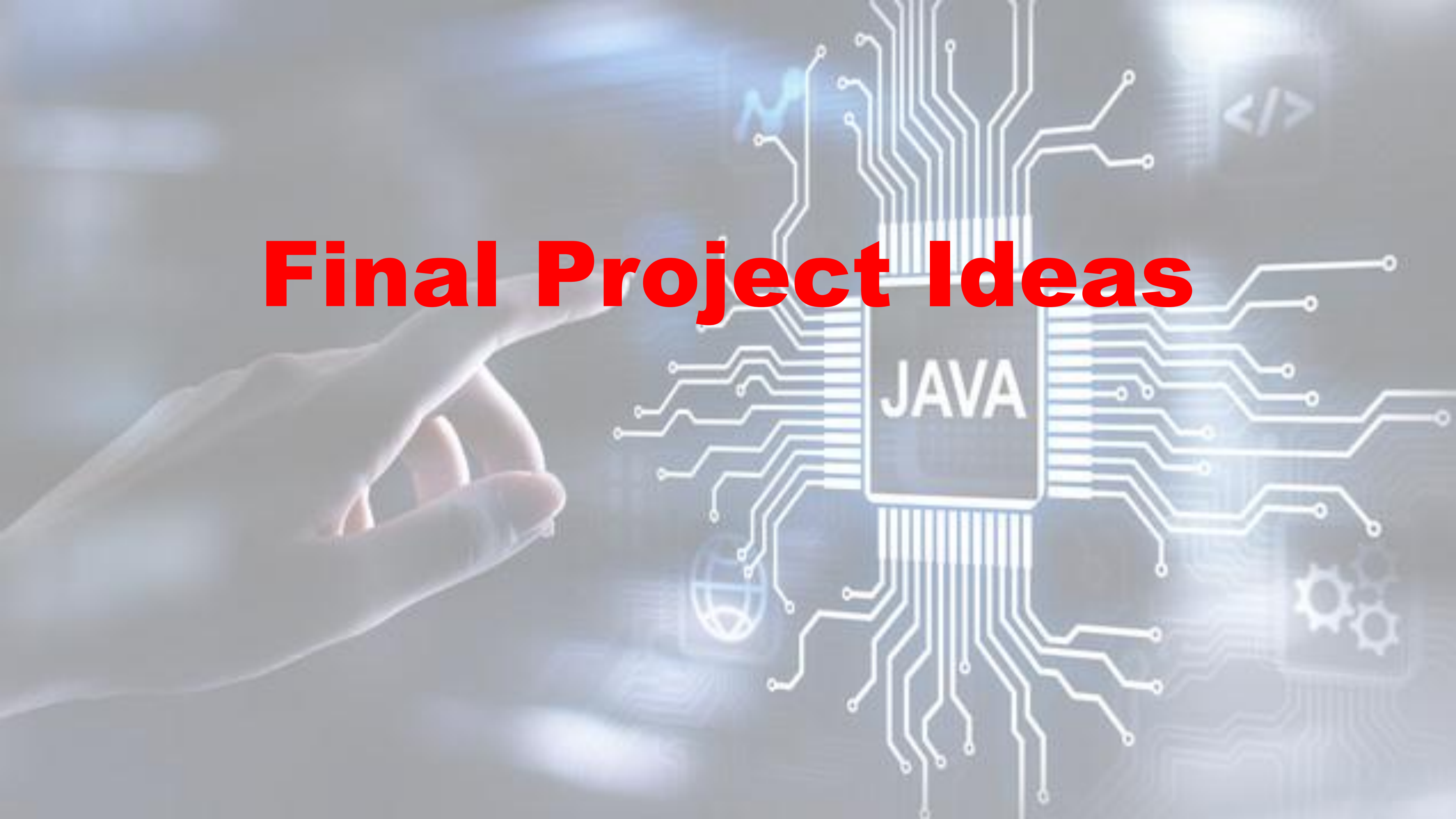
# Syntax

```
private void jTextField2ActionPerformed(java.awt.event.ActionEvent evt) {
    String  answer2 = jTextField2.getText();
    if (answer2.equalsIgnoreCase("It was too tired")) {
        JOptionPane.showMessageDialog(null, "Your Answer is Correct\n"
                + "it has two tires :) ");
       jTextField2.setEnabled(false);
    } else {
        JOptionPane.showMessageDialog(null, "Your Answer is incorrect.\n Try Again!");
        jTextField2.setText("");
    }
}
```

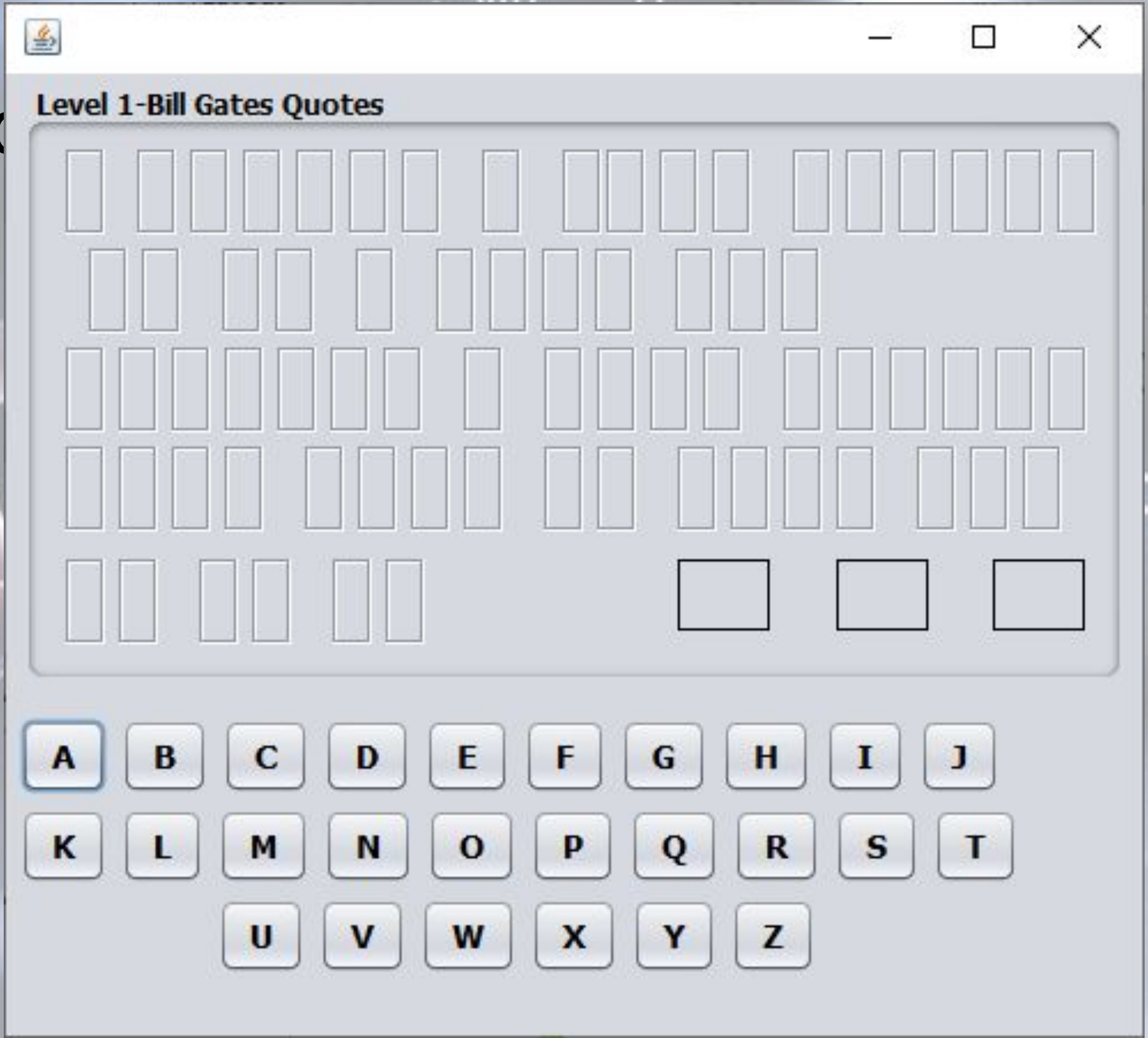
# Syntax

```
private void jTextField3ActionPerformed(java.awt.event.ActionEvent evt) {
    String  answer3 = jTextField3.getText();
    if (answer3.equalsIgnoreCase("Anak")) {
        JOptionPane.showMessageDialog(null, "Your Answer is Correct");
       jTextField3.setEnabled(false);
    } else {
        JOptionPane.showMessageDialog(null, "Your Answer is incorrect.\n Try Again!");
        jTextField2.setText("");
    }
}
```

# Final Project Ideas

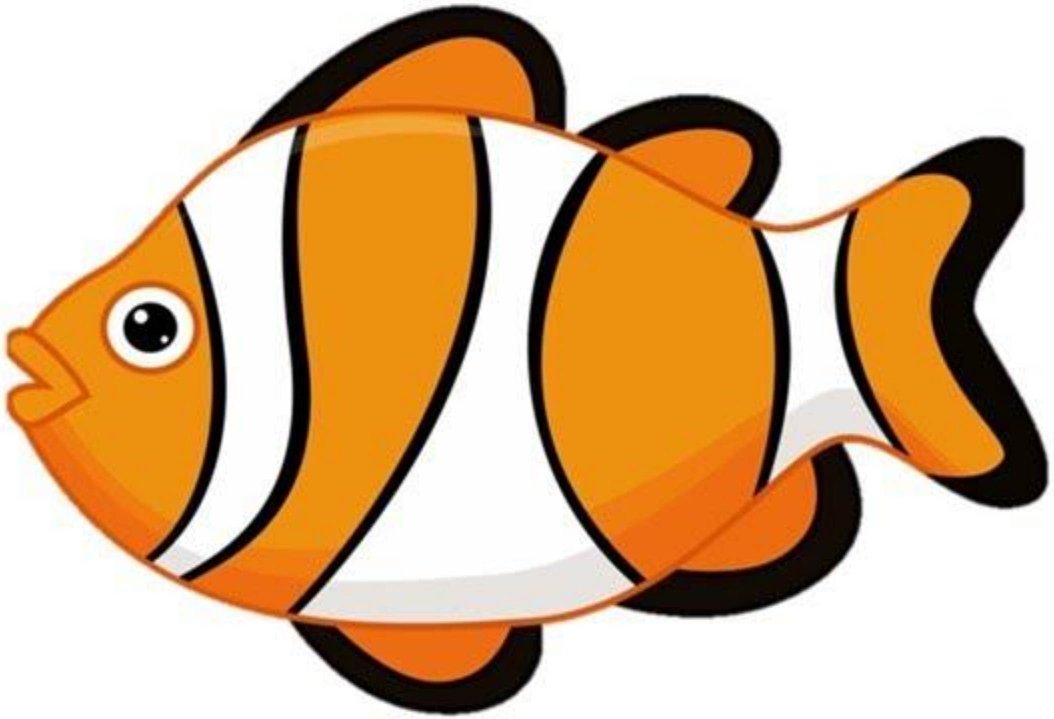


# Guess The Wor

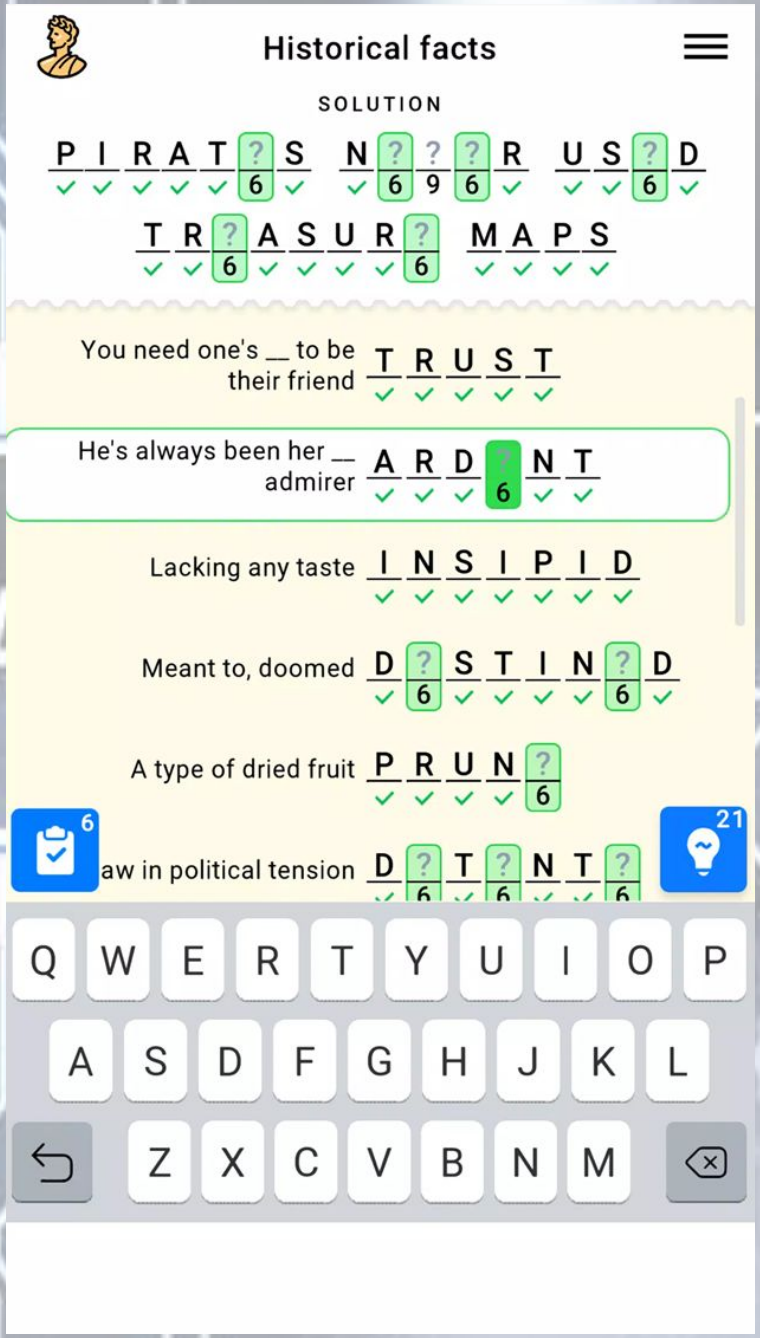


# GUESS THE WORD

+



# Figerrits


Historical facts
SOLUTION
P I R A T ? S N ? ? ? R U S ? D
T R ? A S U R ? M A P S
You need one's __ to be their friend T R U S T
He's always been her __ admirer A R D ? N T
Lacking any taste I N S I P I D
Meant to, doomed D ? S T I N ? D
A type of dried fruit P R U N ?
aw in political tension D ? T ? N T ?
6
21
Q W E R T Y U I O P
A S D F G H J K L
Z X C V B N M

# 1 word 4 pics

# Crossword Puzzle

# Rubrics

# PROGRAM EXECUTION

| Criteria | Excellent (10) | Good (7) | Poor (4) |
|---|---|---|---|
| **Program Execution** | Program executes correctly with no syntax or runtime errors | Program executes with a minor (easily fixed error) | Program does not execute |

# DESIGN LOGIC

| Criteria | Excellent (10) | Good (7) | Poor (4) |
|---|---|---|---|
| **Design Logic** | Program is logically well designed | Program has slight logic errors that does not significantly affect the results | Program is incorrect |

# STANDARD

| Criteria | Excellent (10) | Good (7) | Poor (4) |
| --- | --- | --- | --- |
| **Standards** | The program is followed on the Standard rules or Program Displayed more than Expected | The program followed some Standard rules | Program did not follow Standard rules |

# COOPERATION / COLABORATION

| Criteria | Excellent (10) | Good (7) | Poor (4) |
|---|---|---|---|
| **Cooperation/ Colaboration** | Always contributes to the overall goal of the group | Usually contributes to the overall goal of the group | Rarely contributes to the overall goal of the group |

# RESPONSIVENESS

| Criteria | Excellent (10) | Good (7) | Poor (4) |
|---|---|---|---|
| **Responsiveness** | The response in questions is very satisfied. | The response is quite slow | Didn't response on questions |

# Thank You